

Devoir surveillé

Durée 1h30

*Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées. Dans les portions de code demandées, déclarez les variables utilisées. Toute réponse doit être justifiée.*

Exercice 1 (*Enter the Matrix* (les tubes) - 8 pts)

Nous souhaitons réaliser une application qui crée n fils communiquant entre eux via des tubes anonymes. Nous proposons de créer une paire de tubes pour chaque couple de fils.

- 1°) Avec n fils, combien de tubes sont nécessaires au total ? Combien de descripteurs de fichiers sont nécessaires pour les tubes pour chaque fils ?
- 2°) Pourquoi est-il préférable d'utiliser une paire de tubes entre chaque paire de fils au lieu d'un seul ?
- 3°) Que se passe-t-il si le nombre de fils devient trop important ?
- 4°) Comment un fils peut-il écouter dans l'ensemble de ses tubes ? Proposez une solution en précisant les appels systèmes nécessaires.
- 5°) L'ensemble des tubes est représenté par une matrice de $n \times n$ où chaque case est un tableau de 2 cases. Si on considère que les processus sont numérotés de 0 à $n - 1$, pour le fils i , la ligne i contient les tubes en écriture vers les autres fils et la colonne i contient les tubes en lecture depuis les autres fils. La diagonale n'est pas utilisée.

Donnez la portion de code permettant de créer les tubes et les fils (n est spécifié comme une constante). Les descripteurs inutiles doivent être fermés.
- 6°) Nous souhaitons maintenant laisser la possibilité au père d'écrire des messages à ses fils en utilisant les tubes déjà créés. Expliquez les problématiques et proposez une solution pour les limiter autant que possible, sans créer de nouvelles ressources systèmes.

Exercice 2 (*Éditeur* - 8 pts)

Nous souhaitons développer un éditeur hexadécimal qui permet de visualiser ou de modifier le contenu d'un fichier. L'ensemble des commandes et leurs paramètres est passé en arguments : modification de la valeur d'un octet, affichage d'un bloc du fichier, insertion ou suppression d'un octet, insertion d'un bloc depuis un autre fichier. Vous devez faire en sorte de gérer efficacement les fichiers de grande taille en ne plaçant en mémoire qu'au maximum **MAX** octets (nous supposons que **MAX** est une constante).

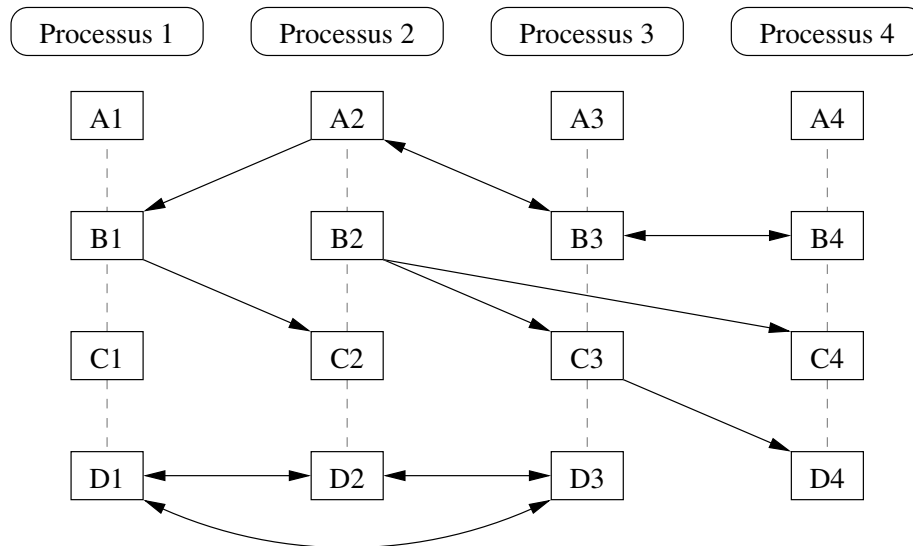
- 1°) Donnez la portion de code permettant d'ouvrir le fichier dont le nom est passé comme premier argument du programme. Faites la gestion d'erreur et précisez à l'utilisateur lorsque le fichier n'existe pas.
- 2°) Nous souhaitons afficher un bloc du fichier. Donnez les arguments nécessaires à passer au programme, puis expliquez (sans donner le code) les différentes actions à réaliser en précisant les appels systèmes nécessaires.
- 3°) Que se passe-t-il si la position demandée est au-delà de la taille du fichier ? Et si le fichier contient moins d'octets que demandés ?

4°) Nous souhaitons supprimer un octet à une position donnée dans le fichier. Écrivez la procédure `supprimer` qui prend en paramètre `fd` le descripteur du fichier et `pos` la position de l'octet à supprimer.

5°) Enfin, nous souhaitons permettre l'insertion d'octets d'un second fichier dans le fichier à une position donnée. Précisez les arguments nécessaires à passer au programme. Expliquez comment procéder en précisant les appels systèmes utilisés. Nous supposons que la taille des données à insérer est inférieure à `MAX`.

Exercice 3 (Sémaphores de Dijkstra - 4 pts)

Soit le diagramme de précedence suivant, représentant 4 processus contenant chacun 4 blocs s'exécutant les uns après les autres :



1°) Proposer une solution à base de sémaphores pour résoudre ces contraintes (1 sémaphore pour chaque cas).

2°) Proposez une solution pour réduire le nombre de sémaphores.

3°) Donnez un diagramme de précedence qui peut induire un interblocage en fonction de l'exécution. Expliquez différentes exécutions possibles.



Annexes : quelques en-têtes de fonctions C

```

void          _exit(int code);
unsigned int  alarm(unsigned int nb_secondes);
int          atexit(void (*procedure)(void));
void         chucknorris(int god);
void         clearerr(FILE *flux);
int          close(int fd);
int          closedir(DIR *repertoire);
int          creat(const char *chemin, mode_t mode);
int          dup(int ancien_fd);
int          dup2(int ancien_fd, nouveau_fd);
int          execve(const char *fichier, char *const argv[], char *const envp[]);
void         exit(int statut);
int          fcntl(int fd, int commande, ...);
int          fclose(FILE *flux);
FILE         *fdopen(int fd, const char *mode);
int          feof(FILE *flux);
int          ferror(FILE *flux);
int          fflush(FILE *flux);
int          fileno(FILE *flux);
FILE         *fopen(const char *chemin, const char *mode);
pid_t        fork();
size_t       fread(void *tampon, size_t taille, size_t nombre_elements, FILE *flux);
FILE         *freopen(const char *chemin, const char *mode, FILE *flux);
int          fseek(FILE *flux, long offset, int point_depart);
int          fstat(int fd, struct stat *tampon);
int          ftell(FILE *flux);
size_t       fwrite(const void *tampon, size_t taille, size_t nombre_elements, FILE *flux);
pid_t        getpid();
pid_t        getppid();
int          kill(pid_t pid, int numero_signal);
off_t        lseek(int fd, off_t offset, int point_depart);
int          lstat(const char *chemin, struct stat *tampon);
void         *memcpy(void *destination, const void *source, size_t taille);
void         *memset(void *tampon, int caractere, size_t nombre_elements);
int          mkfifo(const char *nomFichier, mode_t mode);
int          open(const char *chemin, int options);
int          open(const char *chemin, int options, mode_t mode);
DIR          *opendir(const char *chemin);
int          pause(void);
int          pipe(int tube[2]);
ssize_t      read(int fd, void *tampon, size_t taille);
struct dirent *readdir(DIR *repertoire);
void         rewind(FILE *flux);
void         rewinddir(DIR *repertoire);
void         (*signal(int numero_signal, void (*handler)(int)))(int)
int          sigaction(int numero_signal, const struct sigaction *action,
                      struct sigaction *ancienne_action);
int          sigaddset(sigset_t *ensemble, int numero_signal);
int          sigdelset(sigset_t *ensemble, int numero_signal);
int          sigemptyset(sigset_t *ensemble);
int          sigfillset(sigset_t *ensemble);
int          sigismember(sigset_t *ensemble, int numero_signal);
int          sigpending(sigset_t *ensemble);
int          sigprocmask(int comment, const sigset_t *ensemble, sigset_t *ancien);
int          sigqueue(pid_t pid, int numero_signal, const union sigval valeur);
int          sigsuspend(const sigset_t *ensemble);
int          sigwaitinfo(const sigset_t *ensemble, siginfo_t *info);
int          sigtimedwait(const sigset_t *ensemble, siginfo_t *info,
                        const struct timespec *timeout);
unsigned int  sleep(unsigned int nombre_secondes);

```

```

int          stat(const char *chemin, struct stat *tampon);
time_t       time(time_t *temps);
int          truncate(const char *chemin, off_t offset);
int          ftruncate(int fd, off_t offset);
int          truncate64(const char *chemin, off64_t offset);
int          ftruncate64(int fd, off64_t offset);
int          unlink(const char *nomFichier);
pid_t        wait(int *statut);
pid_t        waitpid(pid_t pid, int *statut, int options);
ssize_t      write(int fd, const void *tampon, size_t taille);

```

Annexes : constantes associées à des paramètres ou des champs de structures

options (open, fcntl) : O_RDONLY, O_WRONLY, O_RDWR, O_CREAT, O_EXCL, O_TRUNC, O_APPEND
mode (creat, open) : S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP, S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH, S_IWOTH, S_IXOTH
commande (fcntl) : F_GETFL, F_SETFL
mode (fopen, fdopen, freopen) : "r", "r+", "w", "w+", "a", "a+"
point_départ (lseek, fseek) : SEEK_SET, SEEK_CUR, SEEK_END
mode (mkfifo) : O_RDONLY, O_WRONLY, O_CREAT, O_EXCL, S_IRWXU, S_IRUSR, S_IWUSR, S_IXUSR, S_IRWXG, S_IRGRP, S_IWGRP, S_IXGRP, S_IRWXO, S_IROTH, S_IWOTH, S_IXOTH
handler (signal, champ sa_handler de struct sigaction) : SIG_IGN, SIG_DFL
sa_flags (champ sa_flags de struct sigaction) : SA_ONESHOT, SA_NOMASK, SA_SIGINFO
comment (sigprocmask) : SIG_BLOCK, SIG_UNBLOCK, SIG_SET
options (waitpid) : WNOHANG, WUNTRACED
valeurs de errno : E2BIG, EACCESS, EAGAIN, EBADE, EBADF, EBADFD, EBADMSG, EADDR, EADDRQC, EBADSLT, EBUSY, ECANCELED, ECHILD, ECHRG, EDEADLK, EDOM, EDQUOT, EEXIST, EFAULT, EFBIG, EHOSTDOWN, EIDRM, EILSEQ, EINPROGRESS, EINTR, EINVAL, EIO, EISDIR, EISNAM, ELOOP, EMEDIUMTYPE, EMFILE, EMLINK, EMSGSIZE, EMULTIHOP, ENAMETOOLONG, ENFILE, ENOBUFS, ENODATA, ENODEV, ENOENT, ENOEXEC, ENOKEY, ENOLCK, ENOLINK, ENOMEDIUM, ENOMEM, ENOMSG, ENOSPC, ENOSR, ENOSTR, ENOSYS, ENOTBLK, ENOTCONN, ENOTDIR, ENOTEMPTY, ENOTSUP, ENOTTY, EOVERFLOW, EPERM, EPFNOSUPPORT, EPIPE, ERANGE, ERESTART, EROFS, EPIPE, ESRCH, ESTALE, ESTRPIPE, ETIME, ETXTBSY, EUCLEAN, EXDEV, EXFULL, ECHUCKTOOSTRONG

Annexes : macros

Sur **statut** de wait, waitpid : WIFEXITED, WEXITSTATUS, WIFSIGNALED, WTERMSIG, WIFSTOPPED, WSTOPSIG
 Sur le champ **st_mode** de struct stat : S_ISREG, S_ISDIR, S_ISFIFO

Annexes : quelques structures C

```

struct stat {
    dev_t      st_dev;
    ino_t      st_ino;
    mode_t     st_mode;
    nlink_t    st_nlink;
    uid_t      st_uid;
    gid_t      st_gid;
    dev_t      st_rdev;
    off_t      st_size;
    blksize_t  st_blksize;
    blkcnt_t   st_blocks;
    time_t     st_atime;
    time_t     st_mtime;
    time_t     st_ctime;
};

struct dirent {
    char d_name[256];
};

struct sigaction {
    void (*sa_handler) (int);
    void (*sa_sigaction) (int, siginfo_t*, void*);
    sigset_t sa_mask;
    int sa_flags;
};

union sigval_t {
    int sival_int;
    void *sival_ptr;
};

struct siginfo_t {
    int si_signo;
    int si_code;
    sigval_t si_value;
    pid_t si_pid;
    ...
};

struct timespec {
    long tv_sec;
    long tv_nsec;
};

```